Experiment 5

Title: Sales Management System using PL/SQL Procedures and Functions

Aim

To implement **Stored Procedures and Functions** in PL/SQL for efficient sales data handling.

Objective

- Create a **procedure** to insert sales records
- Create a **function** to calculate total sales
- Execute and test both

Requirements

Prerequisite: Basic knowledge of SQL, including creating tables, inserting data, and understanding relational databases. Familiarity with SELECT statements and database schemas involving primary and foreign keys.

Hardware Requirement

- Computer with minimum 4GB RAM
- Internet connection

Software Requirement

- MySQL / PostgreSQL
- MySQL Workbench / pgAdmin
- Windows / Linux OS

Theory

Stored Procedure

A **Procedure** is a **stored program** in a database that performs a specific task.

It is written using SQL + programming logic (like PL/SQL in Oracle Database).

Key Points

- Performs actions (INSERT, UPDATE, DELETE)
- Can take input parameters
- **Does NOT return a value directly**
- Used for operations and tasks

- Performs an action
- May or may not return value
- Used for INSERT/UPDATE/DELETE

Function

A **Function** is also a stored program, but it **returns a value**.

Key Points

- Always returns a value
- Used for calculations
- Can be used inside SQL queries
- Accepts parameters

Database Design

Table: Sales

Column	Type	Description
SaleID	NUMBER	Primary Key
ProductName	VARCHAR2	Product
Quantity	NUMBER	Units sold
Price	NUMBER	Price per unit
TotalAmount	NUMBER	Quantity × Price

Steps

Step 1: Create Table

```
CREATE TABLE Sales (
    SaleID NUMBER PRIMARY KEY,
    ProductName VARCHAR2(50),
    Quantity NUMBER,
    Price NUMBER,
    TotalAmount NUMBER
);
```

Step 2: Create Stored Procedure (Insert Data)

Procedure: Add_Sale

```
CREATE OR REPLACE PROCEDURE Add_Sale(
    p_id NUMBER,
    p_name VARCHAR2,
    p_qty NUMBER,
    p_price NUMBER
)
IS
BEGIN
    INSERT INTO Sales
    VALUES (p_id, p_name, p_qty, p_price, p_qty * p_price);

    COMMIT;
END;
/
```

Execute Procedure

```
BEGIN
    Add_Sale(1, 'Laptop', 2, 50000);
    Add_Sale(2, 'Mouse', 5, 500);
    Add_Sale(3, 'Keyboard', 3, 1500);
END;
/
```

Step 3: Create Function (Total Sales)

Function: Get_Total_Sales

```
CREATE OR REPLACE FUNCTION Get_Total_Sales
RETURN NUMBER
IS
    total NUMBER;
BEGIN
    SELECT SUM(TotalAmount)
    INTO total
    FROM Sales;

    RETURN total;
END;
/
```

Step 4: Call Function

```
SELECT Get_Total_Sales AS Total_Sales FROM dual;
```

Sample Output

```
TOTAL_SALES
-----------
108500
```

Result

Successfully created:
Stored Procedure to insert sales data
Function to calculate total sales

Conclusion

The **Sales Management System using PL/SQL Procedures and Functions** was successfully implemented. This practical helped in understanding how **procedures** are used to perform database operations like inserting and managing sales records

Viva Questions

1. Difference between procedure and function?
2. Can a function return multiple values?
3. Why use COMMIT in procedure?
4. Where are stored procedures used in real applications?